

SAHYATRI

Complete Antigravity Build Specification

*Product, UX, architecture, data, APIs, workflows, AI, safety, security, implementation,
and acceptance*

Prepared	June 9, 2026
Baseline	Existing SahYatri repository
Technology	Next.js 16.2.6, React 19.2.4, FastAPI, PostgreSQL
Purpose	Authoritative implementation handoff for an AI coding agent

Journey beyond the map.

SahYatri Antigravity Build Handoff

Version: 1.0 Prepared: 2026-06-09 Implementation baseline: existing SahYatri repository Framework
baseline: Next.js 16.2.6, React 19.2.4, FastAPI, PostgreSQL

Purpose

This folder is the authoritative build specification for an AI coding agent. It translates the project proposal and current repository into testable product, design, architecture, workflow, data, API, AI, security, deployment, and acceptance requirements.

The agent must extend the existing repository. It must not replace the application with a new starter, change the visual identity, remove working features, or introduce infrastructure that is not justified by a requirement.

Reading Order

1. `README.md` - operating rules and authority.
2. `01_PRODUCT_REQUIREMENTS.md` - scope, users, features, roles, and non-functional requirements.
3. `02_UX_UI_DESIGN_SYSTEM.md` - exact visual language, navigation, screens, states, and accessibility.
4. `03_SYSTEM_ARCHITECTURE.md` - target architecture, service boundaries, deployment, jobs, and observability.
5. `04_DATA_API_REALTIME_CONTRACTS.md` - canonical data model, HTTP APIs, Socket.IO events, and environment variables.
6. `05_WORKFLOWS_AI_SECURITY.md` - end-to-end workflows, state machines, algorithms, medical safeguards, and security.
7. `06_IMPLEMENTATION_AND_ACCEPTANCE.md` - execution phases, definition of done, and acceptance tests.
8. `07_CURRENT_STATE_GAP_ANALYSIS.md` - what already works, what is simulated, and what remains.
9. `08_SOURCE_TRACEABILITY.md` - mapping from the supplied PDF to these requirements.

Authority

When instructions conflict, use this order:

1. Safety, privacy, legal, and medical constraints in this pack.
2. Acceptance criteria in `06_IMPLEMENTATION_AND_ACCEPTANCE.md`.
3. Product and workflow requirements in this pack.
4. Existing repository behavior.
5. The supplied academic proposal.

The proposal is vision input, not an exact technical contract. This pack deliberately corrects contradictions and unsafe claims.

Required Agent Behavior

- Read `AGENTS.md` before edits.
- Before changing Next.js code, read the relevant guide in `node_modules/next/dist/docs/`.
- Preserve unrelated user changes and the current dirty worktree.
- Use the App Router and Next.js 16 conventions. Dynamic `params` are promises. `proxy.ts` replaces the old middleware convention.
- Prefer Server Components for data display and Client Components only for interaction, browser APIs, or live state.
- Put authentication and authorization inside every sensitive route and Socket.IO event. Proxy-based redirects are convenience, not the security boundary.
- Validate external input with Zod or Pydantic.
- Keep medical data out of logs, analytics, URLs, client caches, and model prompts.
- Treat emergency guidance as static, professionally reviewed support content. Never present AI output as diagnosis or medical authority.
- Keep the app functional without paid external APIs by using clearly labeled demo fallbacks. Never silently present mock data as live data.
- Add tests with every behavior change. Do not mark a requirement complete without its acceptance test.

Required Deliverable From Antigravity

The completed implementation must include:

- Working web application and FastAPI service.
- Database migrations and deterministic demo seed.
- `.env.example` containing names and descriptions only, never secrets.
- OpenAPI or equivalent API contract.
- Unit, integration, end-to-end, authorization, and critical safety tests.
- Deployment configuration and runbook.
- Updated gap matrix showing no unresolved P0 or P1 requirements.
- A final build report listing changed files, migrations, test results, known limitations, and demo credentials.

Product Definition In One Sentence

SahYatri is a mobile-first travel companion network that helps adults find compatible travel partners, form consent-based groups, collaboratively plan adaptive trips, discover authentic food and places, share limited emergency information, and manage safety and expenses in one coherent workflow.

01. Product Requirements

1. Product Goal

Build a trustworthy, mobile-first travel companion and collaborative trip platform. The core journey is:

register -> complete profile -> discover compatible adults -> chat safely -> form/join group -> plan trip -> execute trip -> handle expenses and safety -> close and review

SahYatri is not a booking agency, medical device, emergency-response provider, or guarantee of companion safety. It provides planning, communication, decision support, and links to third-party services.

2. Target Users

P1 Solo Explorer

An adult who wants companions with similar budget, pace, travel style, food preferences, and risk tolerance.

P2 Group Organizer

Creates a private or match-based group, manages membership, starts a trip, confirms itinerary changes, and closes the trip.

P3 Traveler With Accessibility Or Health Needs

Wants control over what emergency information is shared, reliable reminders, accessible planning, and nearby facility discovery.

P4 Food And Culture Traveler

Prioritizes regional dishes, dietary compatibility, local eateries, verified hygiene information, and hidden cultural experiences.

P5 Platform Operator

Handles moderation, abuse reports, content quality, integration health, model evaluation, and incident response. Admin UI is post-MVP, but operational controls and audit data are required.

3. Scope

MVP, Required

- Credentials authentication and optional Google OAuth.
- Adult-only registration, email normalization, secure password hashing, session handling, sign-out.
- Guided travel profile and optional medical profile.
- Compatibility discovery with explainable sub-scores.
- Anonymous pre-match chat with blocking/reporting controls.
- Private travel groups of 3-5 members and invite-code join.
- Group details, consent controls, and group chat.
- Trip creation and day-by-day itinerary generation.
- Hidden-gem mode for trips of at least 5 calendar days.
- Collaborative itinerary editing with persistence and conflict handling.
- Map, place detail, directions link, public-transport option, and checklist navigation.
- Culinary discovery filtered by budget, dietary restrictions, and evidence-backed hygiene data.
- Expense recording, equal/custom splitting, balances, and settlement suggestions.
- Live location sharing that is opt-in, time-bounded, visible, and revocable.
- Hold-to-activate SOS, emergency contacts, facility lookup, and reviewed first-aid cards.
- Medication schedules and browser push reminders.
- English and Hindi UI architecture; all copy externalized for future languages.
- Free/premium entitlement enforcement through verified payment webhooks.
- Offline shell and read-only access to the active trip's last synchronized itinerary.

MVP+, Planned After Core Acceptance

- Ratings and post-trip reviews.
- Identity verification provider.
- Admin moderation interface.
- Local guide and tourism-board content portal.
- Restaurant reservation and activity-booking commissions.
- Additional languages.
- Redis-based distributed caching and Socket.IO adapter.
- Dedicated worker queue and horizontally scaled realtime nodes.
- Native mobile applications.

Out Of Scope For MVP

- Automatic calls to emergency services.
- Clinical diagnosis, dosage calculation, treatment recommendations, or medical-device claims.

- Background location tracking without an active, explicit sharing session.
- Scraping sources that prohibit scraping.
- Claims of 85% model accuracy, HIPAA compliance, 99.9% SLA, or 10,000 concurrent users without measured evidence.
- MongoDB, RabbitMQ, Kubernetes, or 25 integrations merely because the proposal mentions them.

4. Roles And Permissions

Role	Capabilities
Visitor	View marketing page, register, sign in, read public policies
Free user	Complete profile, discover limited matches, one active group, 3-day trip generation, basic chat and safety
Premium user	More active groups, longer trips, advanced hidden-gem and AI features, richer culinary results
Group creator	Edit group metadata, invite members, initiate trip generation, dissolve group before active trip
Group member	Read group, chat, consent to medical overview, edit collaborative plan according to group policy
Operator	Investigate reports, suspend accounts, review audit events; no routine access to decrypted medical content

Every server action, route handler, and socket event must independently enforce the role and resource membership.

5. Functional Requirements

FR-AUTH: Identity And Account

- FR-AUTH-01: Registration requires name, valid normalized email, password of at least 8 characters, age confirmation of 18+, privacy acceptance, and terms acceptance.
- FR-AUTH-02: Duplicate email returns a generic conflict response without exposing account details.
- FR-AUTH-03: Passwords use an adaptive password hash. Passwords and OAuth tokens never enter logs.
- FR-AUTH-04: Login, logout, expired session, and unauthorized access have explicit UI states.
- FR-AUTH-05: Account settings support export request, consent history, and deletion request. Full self-service completion can be MVP+ but the data model and service boundary are required now.

FR-ONB: Onboarding

- FR-ONB-01: Profile completion is a resumable wizard, not one oversized form.

- FR-ONB-02: Steps are identity, travel preferences, Big Five assessment, food and dietary preferences, interests/languages, safety profile, consent review.
- FR-ONB-03: Medical details are optional. The user may complete onboarding with "No medical information to add."
- FR-ONB-04: Each medical category has a separate consent explanation and can be edited or removed later.
- FR-ONB-05: Profile completion updates the authenticated session and redirects to the dashboard.

FR-MATCH: Discovery And Matching

- FR-MATCH-01: Show only onboarded, active, adult users excluding self, blocked users, and users outside visibility settings.
- FR-MATCH-02: Each card shows name, age, avatar, nationality, travel style, common interests, overall compatibility, and a plain-language explanation.
- FR-MATCH-03: Expose personality, budget, travel style, food, and risk-tolerance sub-scores. Do not expose raw medical conditions.
- FR-MATCH-04: Medical information must not be used to discriminate or hide a user. Show only a consent-based "emergency-sharing ready" indicator.
- FR-MATCH-05: A user can view profile, start pre-match chat, dismiss, block, or report.
- FR-MATCH-06: Results are paginated and deterministically sorted by score, then trust signals, then stable user ID.

FR-CHAT: Pre-Match And Group Chat

- FR-CHAT-01: Pre-match chat hides contact information and exact location by default.
- FR-CHAT-02: Only the two participants can join, fetch, and send to a pre-match chat.
- FR-CHAT-03: Group chat is available only to current group members.
- FR-CHAT-04: Message length, rate, attachment type, and attachment size are limited.
- FR-CHAT-05: Users can block/report from chat. Report preserves required evidence under restricted access.
- FR-CHAT-06: Sentiment/toxicity indicators are advisory moderation signals and are never shown as a definitive judgment of a person.

FR-GRP: Groups

- FR-GRP-01: Groups contain 3-5 members for match-made travel; private groups may begin with one creator while forming.
- FR-GRP-02: Group states are forming, planning, active, completed, dissolved.
- FR-GRP-03: Invite codes are random, unique, revocable, and rate-limited.
- FR-GRP-04: Joining is atomic and cannot exceed maximum size.

- FR-GRP-05: Group details include members, roles, trip summary, compatibility summary, consent readiness, chat, and actions.
- FR-GRP-06: Leaving, removal, and dissolution rules must protect active trip history and financial records.

FR-TRIP: Planning

- FR-TRIP-01: A group member can create a trip with destination, start/end dates, currency, total budget, pace, mobility needs, and priorities.
- FR-TRIP-02: End date must not precede start date. Duration is inclusive calendar days and uses the destination timezone.
- FR-TRIP-03: Generation produces 2-5 realistic items per day, travel/rest buffers, meal windows, costs, durations, place evidence, and alternatives.
- FR-TRIP-04: Trips under 5 days prioritize essential attractions and feasible routing.
- FR-TRIP-05: Trips of at least 5 days activate hidden-gem mode and target 30-40% verified lesser-known experiences. This is a target, not a reason to insert fabricated places.
- FR-TRIP-06: Regeneration creates a version. It never silently deletes the prior itinerary.
- FR-TRIP-07: External place lookup failure retains a draft item labeled unverified and requests user confirmation.
- FR-TRIP-08: Dates, costs, opening hours, and route times show source and freshness.

FR-COLLAB: Collaborative Planning

- FR-COLLAB-01: Members can add, edit, reorder, vote, and remove itinerary items.
- FR-COLLAB-02: Updates are persisted before success is broadcast.
- FR-COLLAB-03: Every write carries an itinerary version. Stale writes receive a conflict response and current version.
- FR-COLLAB-04: Activity history records actor, action, item, timestamp, and version without storing sensitive medical text.

FR-DISC: Places, Food, And Navigation

- FR-DISC-01: Food results show regional specialty, venue, estimated price, dietary compatibility, distance, opening state, rating source, hygiene source, and verification status.
- FR-DISC-02: Unknown hygiene data is displayed as unknown, never invented.
- FR-DISC-03: Hidden-gem cards show why the place qualifies, source evidence, accessibility notes, best visit time, and confidence.
- FR-DISC-04: Navigation offers walking, driving, and transit when available, plus a deep link to the mapping provider.
- FR-DISC-05: Public toilets, pharmacies, clinics, and hospitals are distinct facility filters.

- FR-DISC-06: Checklist navigation supports complete/uncomplete, next stop, skip, and offline read-only state.

FR-BUD: Expenses

- FR-BUD-01: Record payer, amount in minor currency units, currency, category, description, date, optional receipt, and split.
- FR-BUD-02: Equal split includes a deterministic remainder rule. Custom splits must sum exactly to total.
- FR-BUD-03: Only current trip members can be included.
- FR-BUD-04: Balances and settlement suggestions are derived, reproducible, and never use floating-point money math.
- FR-BUD-05: Expense edits and deletion are auditable.

FR-SAFE: Safety And Medical

- FR-SAFE-01: Medical data is encrypted with authenticated encryption and decrypted only server-side for an authorized purpose.
- FR-SAFE-02: Group sharing requires both profile-level sharing and group-specific consent.
- FR-SAFE-03: Group members see only consented emergency categories and instructions, not full history or unrelated notes.
- FR-SAFE-04: SOS uses a three-second hold, visible cancellation, current location confirmation, and fallback when location is denied.
- FR-SAFE-05: SOS creates an incident, broadcasts to the group, looks up nearby facilities, and attempts contact notifications with per-contact status.
- FR-SAFE-06: The UI always tells users to call the local emergency number when urgent.
- FR-SAFE-07: First-aid content is versioned, source-attributed, reviewed, and non-generative.
- FR-SAFE-08: Medication reminders include timezone, recurrence, snooze, taken/skipped status, and permission handling.
- FR-SAFE-09: Live location sharing defaults off, expires automatically, and shows a persistent indicator while active.

FR-PAY: Premium

- FR-PAY-01: Client calls cannot directly grant premium.
- FR-PAY-02: Entitlements change only after verified Stripe/Razorpay events or an operator action.
- FR-PAY-03: Webhooks are idempotent and store provider event IDs.
- FR-PAY-04: Cancellation, failed renewal, grace period, and downgrade behavior are explicit.

FR-I18N: Language And Accessibility

- FR-I18N-01: User-facing strings are externalized; English is complete and Hindi is the first translated locale.
- FR-I18N-02: Dates, currency, numbers, and units use locale-aware formatting.
- FR-I18N-03: User-generated content translation is opt-in and labeled as machine translated.
- FR-I18N-04: The application meets WCAG 2.2 AA for core flows.

6. Non-Functional Requirements

- NFR-01 Performance: p75 LCP under 2.5 seconds on a mid-tier mobile device and 4G for core pages.
- NFR-02 API: p95 internal API latency under 500 ms excluding external AI/map calls; show progress for longer operations.
- NFR-03 Reliability: retries use bounded exponential backoff and idempotency where writes can repeat.
- NFR-04 Security: OWASP ASVS-inspired controls, secure headers, input validation, resource authorization, rate limiting, secret rotation, and audit events.
- NFR-05 Privacy: data minimization, purpose limitation, retention schedule, export/deletion process, and explicit consent history.
- NFR-06 Observability: structured logs, request correlation IDs, health checks, metrics, and external integration status without sensitive payloads.
- NFR-07 Compatibility: latest two versions of Chrome, Edge, Firefox, and Safari; responsive from 320 px upward.
- NFR-08 Maintainability: TypeScript strict mode, documented service boundaries, no duplicated business rules across UI and routes.
- NFR-09 Testing: at least 80% coverage on critical domain modules; coverage percentage is secondary to required scenario tests.
- NFR-10 Data integrity: database constraints and transactions enforce membership, group capacity, itinerary versions, and financial totals.

7. Success Metrics

Metrics are targets to measure, not claims:

- Profile completion rate.
- Match card to conversation conversion.
- Conversation to group conversion.
- Trip generation completion and edit rate.
- Percentage of generated places verified by a provider.

- Hidden-gem save and visit-confirmation rate.
- Expense settlement completion.
- SOS drill completion time and notification success.
- Safety report rate, block rate, and moderation response time.
- SUS score and task completion for core flows.

02. UX And UI Design System

1. Design Direction

Name: **The Intelligent Travel Journal**

The product should feel like a refined field journal: warm paper, editorial photography, quiet map references, tactile cards, and calm typography. It must still behave like a serious safety and planning tool. Decorative motion belongs mainly on marketing surfaces; authenticated workflows prioritize speed, clarity, and accessibility.

Do not redesign SahYatri as a generic blue SaaS dashboard, neon AI product, glass-only interface, or dense enterprise admin panel.

2. Design Tokens

These values are authoritative and match the existing application.

Color

Token	Value	Use
primary	#8F361D	Main actions, brand headings, active navigation
primary-container	#AF4D32	Hover/selected primary surfaces
secondary	#865300	Culinary and discovery labels
secondary-container	#FDB55C	Warm highlight, premium accents
tertiary	#435848	Success, verified, nature
tertiary-container	#5B705F	Tertiary hover
background	#FBF9F4	Main paper background
surface-lowest	#FFFFFF	Inputs and elevated content
surface-low	#F5F3EE	Soft cards
surface	#FBF9F4	Base surface

Token	Value	Use
surface-container	#F0EEE9	Forms and grouped cards
surface-high	#EAE8E3	Hover and inactive controls
surface-highest	#E4E2DD	Footer and strongest neutral panel
on-surface	#1B1C19	Primary text
on-surface-variant	#56423D	Secondary text
outline	#89726C	Muted text and controls
outline-variant	#DDC0B9	Dividers and borders
danger	#BA1A1A	SOS, destructive actions, severe errors
danger-container	#FFDAD6	Error background
success-container	#D1E9D3	Success background

Color must not be the only state indicator. Every status also needs text and/or an icon.

Typography

- Editorial font: EB Garamond, weights 400-700.
- Interface font: Manrope, weights 300-800.
- Display/H1: EB Garamond, 48-80 px marketing, 36-48 px application, line-height 1.05-1.15.
- H2: EB Garamond, 28-36 px.
- H3: EB Garamond or Manrope semibold, 20-26 px.
- Body: Manrope, 16 px, line-height 1.55-1.65.
- Compact body: Manrope, 14 px, line-height 1.5.
- Label: Manrope 11-12 px, weight 600, uppercase, letter spacing 0.08-0.12 em.
- Minimum rendered body text: 14 px. Safety instructions: at least 16 px.

Geometry And Spacing

- Base spacing unit: 4 px.
- Common gaps: 8, 12, 16, 24, 32, 48, 64 px.
- Max application content width: 1200 px.
- Forms: 760-900 px.
- Text measure: 55-75 characters.
- Radius: 8 px controls, 12 px compact cards, 16 px standard cards, 24 px hero/mobile sheets, full pill only for tags/actions.
- Border: 1 px outline-variant at 30-60% opacity.
- Focus ring: 3 px outside ring using primary with high contrast.

- Tactile shadow: 0 10px 30px -5px rgba(143,54,29,.08), 0 4px 12px -2px rgba(86,66,61,.04).

Motion

- Micro transitions: 150-250 ms.
- Drawer/sheet: 250-350 ms.
- Marketing reveals: up to 700 ms.
- Respect `prefers-reduced-motion`; remove parallax, tilt, pulsing, and nonessential transforms.
- Never animate emergency text, form validation, or content layout in a way that delays action.

3. App Shell

Desktop

- Fixed 64 px header with menu trigger left, centered wordmark, profile/auth right.
- Left drawer is 320 px and contains Home, Dashboard, Discover, Groups, Itinerary, Safety, Profile/Preferences, Billing, Settings, and Sign out.
- Main content begins below header with 24-64 px horizontal padding.
- Contextual group/trip switcher appears at the top of authenticated feature pages.

Mobile

- Header remains 56-64 px.
- Bottom navigation contains Dashboard, Discover, Trip, Safety, and Profile.
- Labels are always visible; icons alone are insufficient.
- Bottom sheets are preferred for filters and item details.
- Content reserves space for bottom navigation and safe-area insets.

4. Shared Component Contract

Every data-driven component must implement:

- Loading skeleton matching final geometry.
- Empty state explaining why it is empty and the primary next action.
- Inline recoverable error with retry.
- Permission-denied state without leaking resource existence.
- Offline/stale indicator where applicable.
- Success confirmation for writes.
- Disabled and submitting states that prevent duplicate actions.

Core components:

- AppHeader, SideDrawer, BottomNav, ContextSwitcher.
- PrimaryButton, SecondaryButton, TextButton, DangerButton.
- TextField, Select, Checkbox, RadioGroup, Range, DateRange, TagInput.
- JournalCard, MetricCard, MatchCard, MemberCard, TripCard, PlaceCard, ExpenseRow.
- StatusBadge, SourceBadge, ConfidenceBadge, PremiumBadge.
- Modal, BottomSheet, Toast, AlertBanner, ConfirmDialog.
- Map, MapMarker, RouteSummary, FacilityFilter.
- ConsentPanel, EmergencyCard, HoldToSOS.

5. Information Architecture

Public:

- / Marketing journal.
- /auth/register
- /auth/login
- /privacy
- /terms
- /safety-disclaimer

Authenticated:

- /dashboard
- /discover
- /profile/[id]
- /groups/[groupId]
- /groups/[groupId]/chat
- /itinerary?groupId=...
- /trips/[tripId]
- /trips/[tripId]/expenses
- /safety?groupId=...
- /onboarding
- /settings/profile
- /settings/privacy
- /settings/billing

The existing routes may remain while new detail routes are added incrementally.

6. Screen Specifications

S01 Marketing Home

Goal: explain the value and route visitors to registration.

- Hero: vintage map texture, restrained compass, editorial travel imagery, headline "Journey beyond the map."
- Primary CTA: "Begin your journey"; secondary CTA: "See how it works."
- Sections: matching, collaborative planning, hidden gems, culinary discovery, safety, testimonial, final CTA.
- Do not fetch authenticated data.
- Remote images must have stable sources, dimensions, fallbacks, and appropriate Next image configuration.

S02 Registration

- Two-column desktop: form and quiet editorial image/quote. Single column mobile.
- Fields: name, email, password, confirm password, date of birth or explicit 18+ confirmation, terms, privacy.
- Password requirements appear before submit.
- Success signs in or routes to login, then onboarding.

S03 Login

- Email/password, Google when configured, forgot-password placeholder only if recovery is implemented.
- Error messages are generic for invalid credentials.
- Redirect authenticated users to requested safe return URL or dashboard.

S04 Onboarding Wizard

Progress: 1 Identity -> 2 Travel -> 3 Personality -> 4 Food -> 5 Safety -> 6 Review.

- Autosave each completed step.
- Back/next buttons remain stable.
- Personality sliders show human-readable endpoints, not diagnostic labels.
- Medical step is clearly optional and has "Skip for now."
- Review shows exactly what is public, group-visible with consent, and private.
- Completion requires required travel fields, not medical disclosure.

S05 Dashboard

- Greeting and concise readiness status.
- Metrics: active groups, upcoming trips, profile completion.
- Primary cards: continue active trip, discover companions, create/join group.
- Active group list with status, members, destination, next action.
- Upcoming itinerary with next item and weather/source freshness.

- Safety readiness checklist without exposing conditions.

S06 Discover

- Filter drawer: destination interest, dates, budget, style, group size, languages, interests.
- Match cards show photo, basic identity, compatibility ring, top shared traits, trust/verification, and actions.
- "Why this match?" opens a breakdown with plain-language factors.
- Never show medical conditions, neuroticism labels, or a definitive "safe/unsafe person" score.
- Empty state explains how widening filters affects results.

S07 Profile Detail

- Hero card with avatar, name, age, nationality, travel style, languages.
- Compatibility breakdown and shared interests.
- Food/dietary compatibility.
- Trust signals are evidence-based: verified email, completed profile, completed trips after reviews exist.
- Actions: start anonymous chat, invite to group, block, report.

S08 Groups

- Group header: name, state, invite code management, member count.
- Member grid with creator/member role and consent-ready status.
- Tabs: Overview, Chat, Plan, Expenses, Safety.
- Group state and allowed next actions are prominent.
- Creator controls use confirmation dialogs and server authorization.

S09 Pre-Match Chat

- Anonymous identity presentation until both users choose to reveal.
- Message composer, timestamps, delivery state, block/report.
- Advisory moderation banner only when threshold is crossed; wording: "This conversation may need attention."
- Never expose raw sentiment scores to users.

S10 Trip Setup And Generation

- Setup card fields: group, destination autocomplete, dates, currency, total budget, pace, mobility, priorities.
- Before generation, show a summary and estimated wait.
- Progress stages: validating, generating structure, verifying places, saving.
- Failure preserves inputs and offers retry or save draft.

- Generated result opens the itinerary, not a blank confirmation.

S11 Itinerary

- Header: destination, date range, group, budget status, hidden-gem mode.
- Desktop: day rail left, timeline center, map/details right. Mobile: day tabs and map toggle.
- Day contains ordered cards with time, type, duration, cost, verification, travel buffer, complete state.
- Item detail includes description, source, opening hours, accessibility, dietary notes, directions, vote/edit actions.
- Hidden gems use a warm gold badge and an evidence explanation.
- Regenerate opens scope choices: one item, one day, or full trip; always versions changes.

S12 Culinary Explorer

- Regional dish carousel followed by venue results.
- Filters: dietary needs, budget, venue style, distance, open now, verified hygiene.
- Result copy format: "Try [dish] at [venue], approximately [price] per person."
- Unknown hygiene is neutral gray and says "No verified hygiene source."

S13 Expenses

- Summary: spent, budget remaining, personal balance.
- Add expense sheet supports receipt, payer, category, date, and split.
- Custom split shows remaining amount in real time and cannot submit until exact.
- Settlement list uses names and formatted currency.

S14 Safety

- Top warning: "For immediate danger, call local emergency services."
- Three-second hold SOS centered and visually separated from normal actions.
- Before activation show location and selected group; after activation show incident status, notification results, and facilities.
- Medical card shows the signed-in user's data. Group overview shows only consented categories.
- First-aid cards are static, source/version labeled, large type, and printable.
- AI safety chat, if retained, must be labeled general information and must refuse diagnosis or dosage changes.

S15 Settings And Privacy

- Public profile visibility.
- Blocked users.
- Group medical consent history and revoke controls.

- Active location-sharing sessions.
- Notification permissions and schedules.
- Data export and deletion request.
- Billing and entitlement status.

7. Content Style

- Calm, direct, and human.
- Use "travel companion," not "stranger" once a connection is formed.
- Avoid "perfect match," "guaranteed safe," "AI doctor," or "medically compatible."
- Distinguish "live," "estimated," "cached," "demo," and "unverified."
- Errors explain recovery: what happened, what was preserved, and what to do next.

8. Accessibility Checklist

- Semantic landmarks and one H1 per page.
- Keyboard access for drawer, dialogs, tabs, cards, reorder controls, maps, and SOS.
- Visible focus on all controls.
- Minimum 44x44 px touch targets.
- Form labels, descriptions, and inline error association.
- `aria-live` for generation, chat delivery, location state, and SOS progress.
- Map information duplicated in a list.
- Charts and score rings include text values.
- Contrast meets WCAG 2.2 AA.
- Reduced-motion mode is fully usable.
- Screen reader confirmation before and after SOS activation.

03. System Architecture

1. Architectural Decision

Use a modular monolith plus one ML service for MVP:

- Next.js 16.2.6 App Router: UI, authentication, route handlers, domain services, database access, payment webhooks.
- Custom Node HTTP server: hosts Next.js and Socket.IO.
- FastAPI: isolated numeric/ML inference endpoints.
- PostgreSQL: authoritative transactional datastore.
- Browser service worker: offline shell and active-trip cache.

- External providers: maps/places, weather, translation, payments, notifications, and optional OpenAI.

Do not add MongoDB, Redis, RabbitMQ, Kubernetes, or separate services until scale or reliability measurements justify them.

```

flowchart LR
    U["Web/PWA client"] --> N["Next.js 16 application"]
    U <--> S["Socket.IO gateway"]
    N --> P["PostgreSQL"]
    N --> M["FastAPI ML service"]
    N --> X["External providers"]
    S --> P
    W["Scheduled workers"] --> P
    W --> X
    N -. same Node process for MVP .- S

```

2. Layer Boundaries

Presentation

src/app, src/components, src/hooks

- Pages compose features.
- Client components own browser interactions only.
- No direct database, secret, or payment-provider access.

HTTP And Realtime Adapters

src/app/api, server.ts

- Authenticate.
- Validate request.
- Authorize resource.
- Call domain/service function.
- Map domain result to stable response.
- Do not contain duplicated business formulas.

Domain

src/lib

- Matching, group rules, itinerary versioning, expense math, consent, emergency orchestration.
- Pure functions where possible.
- Money uses integer minor units or a decimal library.

Integrations

src/services

- Provider-specific clients, timeouts, retries, circuit state, response normalization, and source metadata.
- Providers never leak their response shape into UI components.

Persistence

src/db

- Drizzle schema, migrations, repositories, and transaction boundaries.
- Database constraints mirror domain invariants.

ML

python-service

- Pydantic request/response models.
- Versioned inference metadata.
- Deterministic fallbacks are identified as heuristics.
- Training is outside the web request lifecycle.

3. Next.js 16 Rules

- Read the local guide in `node_modules/next/dist/docs/` before edits.
- Use `proxy.ts`; middleware terminology and file convention are deprecated.
- Proxy performs route redirection only. Sensitive APIs still call `auth()` and resource authorization.
- Dynamic route `params` are asynchronous and must be awaited.
- Route handlers use `Web Request/Response` or Next request/response APIs.
- Server Components fetch server-side data directly through domain/repository functions when practical; do not call the app's own HTTP endpoint from the server without a boundary reason.
- Client Components are limited to forms, sockets, maps, browser location, uploads, and interactive state.
- `loading.tsx`, `error.tsx`, and `not-found.tsx` are required for major route groups.
- Cache authenticated or rapidly changing data explicitly as `no-store`. Cache provider reference data with documented TTL and tags.
- A custom server removes some platform optimizations; deployment must use a persistent Node runtime, not a pure serverless target.

4. Request Pipeline

1. Correlation ID is accepted or generated.
2. Proxy redirects unauthenticated page navigation.
3. Route handler authenticates.

4. Zod validates path/query/body.
5. Authorization checks ownership/membership/role.
6. Domain service executes inside a transaction if multiple writes are related.
7. Audit event is recorded for sensitive actions.
8. Response uses the standard envelope.

Standard success:

```
{"data": {}, "meta": {"requestId": "...", "source": "live", "generatedAt": "..."}}
```

Standard error:

```
{"error": {"code": "GROUP_FULL", "message": "This group is already full.", "requestId": "...", "details": []}}
```

5. Realtime Architecture

Socket handshake must authenticate from the session cookie or a short-lived signed socket token. Client-supplied `userId` is never trusted.

Rooms:

- `user:{userId}`
- `group:{groupId}`
- `prematch:{chatId}`
- `trip:{tripId}`

For each event:

1. Validate payload.
2. Derive user from socket identity.
3. Authorize room/resource.
4. Persist durable writes.
5. Broadcast the persisted record and version.
6. Acknowledge with success or structured error.

Location events are ephemeral by default. The database stores sharing-session metadata and SOS snapshots, not a continuous location history unless the user explicitly opts in for a defined purpose.

6. Background Work

MVP jobs may run as separate Node processes using database-backed leases:

- Weather/traffic/event refresh for active trips every 15 minutes.
- Medication reminders every minute with idempotency key.
- Notification retries.

- Expired location-session cleanup.
- Provider cache cleanup.
- Data-retention/deletion tasks.

Do not run cron loops inside every horizontally scaled web instance without a distributed lease.

7. External Provider Strategy

Every provider client implements:

- 3-10 second timeout based on operation.
- Bounded retries only for transient/idempotent calls.
- Rate-limit awareness.
- Normalized domain response.
- Source, fetched-at, and confidence/verification metadata.
- Clearly labeled fallback.
- Health metric and error classification.

Fallback hierarchy:

- Places/maps: configured provider -> OpenStreetMap/Nominatim within usage policy -> unverified draft.
- Itinerary: configured LLM -> deterministic local template -> manual draft.
- Weather/traffic/events: live provider -> stale cache -> unavailable label. Never fabricate live conditions.
- Notifications: provider -> queued retry -> visible failure status.

8. Data And Caching

- PostgreSQL is the source of truth.
- In-process cache is allowed only for low-risk development/reference data.
- Provider cache belongs in PostgreSQL for a single-instance MVP.
- Redis becomes required before multi-instance Socket.IO, distributed rate limits, or high-volume provider caching.
- Cache keys include provider, locale, coordinates/query, and relevant filters.
- Medical plaintext is never cached.

9. Security Architecture

- Session cookies: secure, HTTP-only, same-site policy appropriate to OAuth.
- CSRF protection for state-changing browser requests according to Auth.js/Next.js guidance.
- Content Security Policy and restricted external image/connect domains.

- Rate limits for login, registration, invite join, chat, generation, uploads, SOS, and webhooks.
- Server-side MIME validation and object storage for avatars/receipts. Base64 blobs do not belong in the user table.
- Authenticated encryption for medical fields with per-record nonce and key version.
- Secrets supplied at runtime and rotated. No placeholder secret is accepted in production.
- Logs redact authorization headers, cookies, medical data, chat content by default, exact coordinates, and payment payloads.

10. Deployment Topology

Local

Docker Compose:

- web:3000
- ml-service:8000
- postgres:5432

Production MVP

- One or more persistent Node containers behind TLS load balancer.
- One FastAPI deployment.
- Managed PostgreSQL with encrypted storage, point-in-time recovery, and private networking.
- Object storage/CDN for avatars and receipts.
- Scheduled worker deployment.
- Provider secrets in a managed secret store.

Before scaling web above one instance, add sticky sessions or preferably Redis Socket.IO adapter plus shared rate limiting.

11. Observability

Required:

- /api/health/live process health.
- /api/health/ready database and required dependency readiness.
- FastAPI /health.
- Structured JSON logs with request ID, route, status, duration, user hash, and error code.
- Metrics: request count/latency/errors, socket connections/events/errors, provider latency/errors, generation duration, job lag, notification status.
- Audit events: login security events, consent change, medical read, group membership change, itinerary version, expense mutation, SOS, entitlement change, export/deletion.

- Error tracking with source maps and sensitive-data scrubbing.

12. Architecture Decision Records

Create ADRs for:

- Custom Socket.IO server.
- PostgreSQL-only MVP.
- Medical encryption scheme.
- Matching formula and fairness controls.
- LLM itinerary generation and verification.
- Realtime location retention.
- Payment entitlement source of truth.

04. Data, API, And Realtime Contracts

1. Data Conventions

- IDs: UUID.
- Timestamps: timezone-aware UTC in storage; locale/destination timezone in UI.
- Money: integer minor units plus ISO 4217 currency. Migrate existing *real* monetary columns.
- Coordinates: validated latitude/longitude; exact coordinates are sensitive.
- Soft deletion: accounts, groups with history, and moderated content use status/deleted timestamp.
- All mutable records have `created_at`, `updated_at`; concurrent resources also have integer `version`.
- Sensitive reads and writes create audit events.
- JSON is used only for genuinely variable provider metadata, not as a substitute for modeled relationships.

2. Existing Core Tables

The following tables exist and remain part of the target model.

``users``

`id`, unique normalized `email`, nullable `password_hash`, `name`, `avatar_url`, `age`, `gender`, `nationality`, `auth_provider`, `is_verified`, `is_onboarded`, `role`, `created_at`, `updated_at`.

Target additions: status, locale, timezone, date_of_birth or auditable adult confirmation, terms_accepted_at, privacy_accepted_at, deleted_at. Avatar becomes an object-storage URL, not a data URL.

`personality\_profiles`

id, unique user_id, Big Five values openness, conscientiousness, extraversion, agreeableness, neuroticism, travel_style, risk_tolerance, budget_level, preferred_group_size, languages_spoken, food_preferences, interests, embedding_vector, completed_at, updated_at.

Values are validated: traits 0-1, risk 1-5, group size 3-5.

`medical\_profiles`

id, unique user_id, blood_type, encrypted conditions, medications, allergies, emergency contacts, first-aid notes, non-sensitive condition_categories, share_with_group, timestamps.

Target additions: encryption_key_version, per-field nonce/tag or encrypted envelope, profile_version, reviewed_at. share_with_group is a profile-level master switch; group consent remains separate.

`travel\_groups`

id, name, status, max_members, destination/date hints, created_by, compatibility_score, unique invite_code, timestamps.

Target additions: visibility, invite_code_expires_at, invite_code_version, version, dissolved_at.

`group\_members`

id, group_id, user_id, role, medical_sharing_consent, joined_at; unique group/user pair.

Target additions: status, left_at, consent_updated_at. Capacity is enforced transactionally.

`compatibility\_scores`

id, canonical ordered user pair, overall and component scores, computed_at.

Target additions: risk_tolerance_score, algorithm_version, explanation, input_profile_versions, expires_at. Scores are invalidated when either profile changes.

`trips`

id, group_id, destination and coordinates, dates, duration, status, budget fields, currency, hidden-gem mode, generation prompt/debug reference, timestamps.

Target additions: destination timezone, pace, mobility needs, priorities, current itinerary version, source mode, generation status. Prompt text must not contain medical data.

`itinerary\_days`

id, trip_id, day_number, date, theme, weather forecast, notes.

Target additions: version_id; unique version/day pair.

`itinerary\_items`

id, day_id, order, type, name, description, location, coordinates, provider place ID, duration, cost, hidden-gem metadata, opening hours, photo, tips, completion state.

Target additions: scheduled local time, verification status, source, fetched-at, accessibility notes, dietary flags, vote summary, revision.

`messages`

id, group_id, nullable sender, content, type, sentiment score, metadata, timestamp.

Target additions: client_message_id, edit/delete timestamps, moderation state. Attachments are separate records.

`expenses`

id, trip_id, payer, amount, currency, description, category, split type, optional JSON splits, timestamp.

Target: use amount_minor, remove duplicate JSON split authority, add expense date, receipt URL, updated/deleted fields, version.

`expense\_splits`

id, expense_id, trip_id, user_id, amount owed, payment state, timestamp.

Target: amount_owed_minor, settled_at; unique expense/user.

`restaurants`

Provider ID, name, cuisine, price/rating, coordinates/address, city/country, specialties, local-gem score, photo/opening data, fetched time.

Target additions: provider, source URL, hygiene value/source/date, verification status, dietary features.

`hidden\_gems`

Provider ID, name/category/description/story, coordinates, city/country, gem score components, endorsements, photo, best time, tips, verification, timestamp.

Target additions: source evidence records, accessibility, confidence, last verified date.

`pre\_match\_chats` And `pre\_match\_messages`

Chat participants, anonymity/status/sentiment/timestamps and messages.

Target additions: reveal state, block/report state, client message ID, moderation state, edit/delete timestamps.

3. Required New Tables

Table	Purpose / Key fields
itinerary_versions	id, trip_id, version, status, created_by, reason, parent_version_id, timestamps
itinerary_votes	unique item_id/user_id, vote, timestamp
audit_events	actor, action, resource type/ID, request ID, redacted metadata, timestamp
consent_events	user, consent type, group, granted/revoked, policy version, timestamp
blocks	blocker, blocked user, reason code, timestamp
reports	reporter, subject/content reference, category, restricted evidence, status, timestamp
location_sharing_sessions	user, group/trip, starts, expires, revoked, purpose
emergency_incidents	initiator, group, coarse/exact encrypted location, status, timestamps
notification_attempts	incident/reminder, channel, recipient reference, provider ID, status, error code
medication_schedules	encrypted medicine details, timezone, recurrence, active state
medication_events	schedule, due time, taken/skipped/snoozed, timestamp
payment_customers	user/provider customer IDs
subscriptions	user, provider, external ID, status, plan, period, grace date
webhook_events	provider event ID, type, received/processed timestamps, result; unique provider/event
provider_cache	provider, key, normalized payload, freshness, expiry
attachments	owner, purpose, object URL/key, MIME, size, scan state
push_subscriptions	user, endpoint and keys encrypted, timestamps

4. API Rules

- Base: /api.
- JSON unless webhook raw body or file upload.

- All list endpoints use cursor pagination: `?cursor=&limit=`.
- `limit` defaults 20, maximum 100.
- Mutations accept `Idempotency-Key` when duplicate execution is harmful.
- Concurrency-controlled writes accept `If-Match` or explicit `version`.
- Validation errors return 400, unauthenticated 401, unauthorized 403, absent 404, conflict 409, rate limit 429.
- Do not return stack traces or provider secrets.

5. Existing Endpoints To Preserve And Harden

Method	Path	Contract
POST	/auth/register	Register adult credentials user
GET/POST	/auth/[...nextauth]	Auth.js handlers
GET/POST	/users/profile	Read/upsert own travel profile
POST	/users/avatar	Replace with multipart/object storage upload
GET/POST	/medical/profile	Read/upsert own decrypted profile; never client-cache
GET	/matching/discover	Cursor-paginated explainable matches
POST	/chat/pre-match	Create/reuse participant chat
GET	/chat/{chatId}/messages	Participant-only history
GET/POST	/groups	List memberships/create group
POST	/groups/join	Atomic invite join
GET	/groups/{groupId}	Member-only details
GET	/groups/{groupId}/messages	Member-only history
POST	/groups/{groupId}/medical-consent	Grant/revoke group-specific consent
GET	/trips	List own trips or group trip
POST	/trips/generate	Start versioned generation
POST	/trips/{tripId}/collaborative-items	Versioned itinerary mutation
GET/POST	/trips/{tripId}/expenses	Member-only expense list/create
GET	/explore/food	Verified food discovery
POST	/emergency	Create emergency incident/orchestration

Method	Path	Contract
POST	/chat/bot	General information only; premium and safety guardrails
GET	/place-photo	Server-side safe provider lookup, no open redirect
POST	/webhooks/stripe	Verified idempotent webhook
POST	/webhooks/razorpay	Verified idempotent webhook

The existing `/users/upgrade` endpoint must be removed or restricted to development-only test fixtures. It must never grant production entitlement from an authenticated client request.

6. Required Additional Endpoints

Account And Privacy

- GET `/account`
- PATCH `/account`
- POST `/account/export`
- DELETE `/account`
- GET `/consents`
- GET `/blocks`
- POST `/blocks`
- DELETE `/blocks/{userId}`
- POST `/reports`

Groups

- PATCH `/groups/{groupId}`
- POST `/groups/{groupId}/invite/rotate`
- POST `/groups/{groupId}/leave`
- DELETE `/groups/{groupId}/members/{userId}`
- POST `/groups/{groupId}/dissolve`

Trips And Itinerary

- POST `/trips`
- GET `/trips/{tripId}`
- PATCH `/trips/{tripId}`
- GET `/trips/{tripId}/versions`
- POST `/trips/{tripId}/regenerate`
- PATCH `/trips/{tripId}/items/{itemId}`

- DELETE /trips/{tripId}/items/{itemId}
- POST /trips/{tripId}/items/{itemId}/vote
- POST /trips/{tripId}/items/{itemId}/complete
- GET /trips/{tripId}/facilities
- GET /trips/{tripId}/routes

Expenses

- PATCH /trips/{tripId}/expenses/{expenseId}
- DELETE /trips/{tripId}/expenses/{expenseId}
- GET /trips/{tripId}/balances
- POST /trips/{tripId}/settlements/{splitId}/mark-paid

Safety

- POST /location-sessions
- DELETE /location-sessions/{sessionId}
- POST /emergency-incidents
- GET /emergency-incidents/{incidentId}
- POST /emergency-incidents/{incidentId}/cancel
- POST /emergency-incidents/{incidentId}/resolve
- CRUD /medication-schedules
- POST /medication-events
- CRUD /push-subscriptions

Billing And Health

- POST /billing/checkout
- GET /billing/subscription
- POST /billing/portal
- GET /health/live
- GET /health/ready

7. Important Request Contracts

Trip generation

```
{
  "groupId": "uuid",
  "destination": {"label": "Goa, India", "placeId": "...", "lat": 15.29, "lng": 74.12,
"timezone": "Asia/Kolkata"},
  "startDate": "2026-12-10",
  "endDate": "2026-12-14",
  "currency": "INR",
  "budgetTotalMinor": 120000,
  "pace": "balanced",
  "mobilityNeeds": ["limited_stairs"],
```

```
{
  "priorities": ["food", "culture", "hidden_gems"]
}
```

Response 202:

```
{"data":{"jobId":"uuid","tripId":"uuid","status":"queued"}}
```

Generation status can be polled or emitted over the trip room.

Expense

```
{
  "description": "Dinner",
  "amountMinor": 425000,
  "currency": "INR",
  "paidBy": "uuid",
  "category": "food",
  "splitType": "custom",
  "splits": [{"userId":"uuid","amountMinor":212500},{ "userId":"uuid","amountMinor":212500}]
}
```

Server verifies exact sum, currency consistency, and membership.

Emergency incident

```
{
  "groupId": "uuid",
  "location": {"lat": 15.29, "lng": 74.12, "accuracyMeters": 18, "capturedAt": "ISO-8601"},
  "incidentType": "unspecified",
  "notifyEmergencyContacts": true
}
```

The server derives user identity. It returns incident ID, local emergency number configuration, facility results, and notification attempt statuses.

8. Socket.IO Contract

All client emits use acknowledgment callbacks and schema version v: 1.

Client To Server

Event	Payload
group:join	{v, groupId}
group:message:send	{v, groupId, clientId, messageId, content, type, metadata?}
prematch:join	{v, chatId}
prematch:message:send	{v, chatId, clientId, messageId, content}
trip:join	{v, tripId}

Event	Payload
itinerary:update	{v, tripId, baseVersion, operation}
location:start	{v, sessionId}
location:update	{v, sessionId, lat, lng, accuracy, capturedAt}
location:stop	{v, sessionId}
emergency:activate	{v, incidentId}

No payload contains a trusted `userId`.

Server To Client

Event	Payload
group:message:new	Persisted message record
prematch:message:new	Persisted message record
itinerary:updated	{tripId, version, operation, actorSummary}
itinerary:conflict	{tripId, expectedVersion, currentVersion}
location:updated	Short-lived member location
location:stopped	Session/member
emergency:activated	Incident summary
emergency:notification-status	Contact/channel status without sensitive contact value
generation:progress	{jobId, stage, percent, message}
error	{code, message, requestId}

9. FastAPI ML Contracts

Retain and version:

- POST `/api/ml/v1/matching/cluster`
- POST `/api/ml/v1/matching/conflict-risk`
- POST `/api/ml/v1/medical/destination-risk`
- POST `/api/ml/v1/itinerary/gem-score`
- POST `/api/ml/v1/chat/moderation-signal`
- POST `/api/ml/v1/culinary/rank`

Every response includes:


```
{"modelVersion":"heuristic-1.0","mode":"heuristic","result":{},"warnings":[]}
```

Use mode: "trained_model" only when a persisted model artifact, training record, evaluation report, and model card exist.

10. Environment Variables

Required production variables:

- DATABASE_URL
- AUTH_SECRET/the exact Auth.js secret variable used by the installed version
- AUTH_TRUST_HOST
- NEXT_PUBLIC_APP_URL
- ML_SERVICE_URL
- MEDICAL_ENCRYPTION_KEY or KMS key configuration
- GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET when OAuth enabled
- GOOGLE_MAPS_SERVER_API_KEY and separately restricted browser key if needed
- OPENAI_API_KEY when LLM generation enabled
- OPENWEATHER_API_KEY
- Stripe key, price ID, and webhook secret
- Razorpay key ID, key secret, and webhook secret
- Twilio credentials when SMS enabled
- Web push VAPID keys
- Object-storage bucket/endpoint credentials
- Observability endpoint/DSN

Rules:

- Public variables contain no secrets.
- Production startup fails for missing required security secrets.
- .env.example documents purpose, format, and whether optional.

05. Workflows, AI, Safety, And Security

1. Account And Onboarding Workflow

```
stateDiagram-v2
    [*] --> Visitor
    Visitor --> Registered: register/OAuth
    Registered --> Onboarding: authenticated
    Onboarding --> Onboarding: save step
    Onboarding --> Ready: required travel profile complete
    Ready --> Suspended: moderation action
    Ready --> DeletionPending: deletion request
```

DeletionPending --> Deleted: retention workflow

Rules:

1. Normalize email and verify adult eligibility.
2. Create account and session.
3. Redirect to the first incomplete onboarding step.
4. Save each step idempotently.
5. Medical step is optional.
6. Review visibility and consent.
7. Mark onboarded and refresh session.
8. Route to dashboard.

2. Matching Workflow

1. Load the user's current profile and visibility settings.
2. Select eligible candidates: onboarded, active, adult, not self, not blocked either direction.
3. Compute component scores using versioned inputs.
4. Apply user filters.
5. Rank and paginate.
6. Return score explanation and algorithm version.
7. Allow view, dismiss, chat, invite, block, report.

Canonical Compatibility Formula

Use the proposal's explainable factors as the target v1 formula:

- Personality alignment: 30%.
- Travel-style alignment: 25%.
- Budget alignment: 15%.
- Food/dietary alignment: 10%.
- Risk-tolerance/pace alignment: 10%.
- Shared interests/language fit: 10%.

Medical conditions are not a compatibility penalty. A separate boolean/label may indicate that both users have enabled emergency-sharing readiness.

Scores:

- Traits: normalized distance with documented interpretation.
- Travel style: matrix, not only exact match; mixed is compatible with multiple styles.
- Budget: exact 100, adjacent 60, opposite 20.
- Food: dietary conflicts are surfaced separately; preference similarity uses Jaccard/cosine.

- Risk: closeness on 1-5 scale.
- Interests/languages: weighted overlap.

The UI must say "compatibility estimate," show factors, and allow feedback.

3. Pre-Match To Group Workflow

1. User starts or reuses a chat with a candidate.
2. Server confirms neither party blocked the other.
3. Socket joins participant-only room.
4. Messages persist before broadcast.
5. Either user may report/block/end chat.
6. User invites candidate to an existing forming group or creates a group.
7. Candidate accepts through an atomic join action.
8. Group becomes `planning` when it has at least three members and a trip is initiated.

Invite acceptance must re-check group state, capacity, user status, block relationships, and duplicate membership inside one transaction.

4. Group State Machine

```
stateDiagram-v2
    [*] --> forming
    forming --> planning: minimum members + trip draft
    forming --> dissolved: creator dissolves
    planning --> active: itinerary confirmed + start reached
    planning --> forming: trip removed / below minimum
    planning --> dissolved: group cancels
    active --> completed: trip closed/end reached
    completed --> [*]
```

Active groups retain membership, messages, expenses, and incident history. Removal during an active trip is an operator-assisted or carefully constrained action.

5. Trip Generation Workflow

1. Validate membership, dates, destination, currency, and budget.
2. Create trip draft and generation job.
3. Aggregate group preferences without sending names, medical content, or unnecessary identifiers to the LLM.
4. Generate a structured day plan.
5. Validate JSON/schema, duration, item counts, meal/rest windows, and hidden-gem target.
6. Verify every place against a provider.
7. Mark each item `verified`, `unverified`, or `provider_unavailable`.
8. Calculate rough route feasibility and costs.

9. Save a new itinerary version transactionally.
10. Mark job complete and notify trip room.

Failures:

- Invalid AI response: one repair attempt, then deterministic fallback.
- Provider lookup failure: preserve item as unverified.
- Database failure: job fails; no partially active version.
- Regeneration: prior version remains available.

6. Dynamic Replanning Workflow

Trigger for active trips every 15 minutes and on manual refresh:

1. Read current version and next 6 hours.
2. Fetch weather, traffic, closures/events with freshness.
3. Identify impacted items.
4. Produce a proposal, not an automatic destructive change.
5. Explain reason, source, impact, and alternatives.
6. Group creator or configured quorum accepts.
7. Save a new version and broadcast.

Severe safety alerts may display immediately, but itinerary mutation still remains explicit unless an operator-configured emergency rule applies.

7. Expense Workflow

1. Validate trip membership and amount.
2. Convert to integer minor units.
3. Validate split members and exact total.
4. Insert expense and splits in one transaction.
5. Recalculate balances from immutable records.
6. Generate greedy settlement suggestions.
7. Mark payments manually for MVP; never claim money was transferred.

Equal split remainder goes one minor unit at a time to members sorted by stable user ID.

8. Live Location And SOS Workflow

Location

1. User selects group and duration: 15 minutes, 1 hour, 8 hours, until trip end.
2. Browser asks permission in response to user action.
3. Server creates sharing session.

4. Persistent UI indicator shows active sharing and expiry.
5. Updates are authorized by session ID and rate-limited.
6. Stop, logout, expiry, or group departure revokes sharing.

SOS

```
sequenceDiagram
    participant U as User
    participant C as Client
    participant A as App
    participant D as Database
    participant P as Providers
    participant G as Group
    U->>C: Hold SOS for 3 seconds
    C->>U: Confirm group and location
    C->>A: Create incident
    A->>D: Persist incident
    A->>P: Find facilities and notify contacts
    A->>G: Broadcast incident
    A-->>C: Incident and attempt statuses
    C->>U: Show emergency number, facilities, status
```

If coordinates are unavailable, allow manual location text and still show local emergency numbers. Do not wait for AI or provider lookup before telling the user to call emergency services.

9. AI And ML Rules

Classification Of Current Algorithms

- K-Means endpoint: runtime clustering, not an accuracy-rated trained classifier.
- Conflict endpoint: feature-distance heuristic, not a Random Forest in the current code.
- Medical risk: rule heuristic.
- Hidden gem: weighted heuristic, not XGBoost in the current code.
- Sentiment: keyword heuristic, not BERT.
- Culinary: cosine ranking.

The product and documentation must identify them honestly until trained artifacts exist.

LLM Itinerary Rules

- Use a current configured model name; do not hard-code an obsolete model without a configuration layer.
- Structured output schema is mandatory.
- Temperature should favor reliability.
- Prompt excludes medical details, contact information, exact home location, and secrets.
- Provider verification is mandatory before a place appears as verified.
- Generated prices/opening hours are estimates unless provider sourced.

- Store prompt template version and response diagnostics, not unnecessary personal prompt content.
- Include deterministic fallback and explicit `sourceMode`.

Model Lifecycle

Before `trained_model` mode:

- Dataset provenance and license.
- Train/validation/test split without leakage.
- Baseline comparison.
- Metrics appropriate to task.
- Fairness slices by relevant demographics.
- Calibration and threshold rationale.
- Model card, version, artifact checksum, rollback.
- Human review for safety-relevant behavior.

No model automatically blocks a user, diagnoses a condition, or triggers emergency treatment.

10. Medical Safety Requirements

- The platform is not a substitute for professional medical advice.
- First-aid content requires named clinical review, source, jurisdiction, effective date, review date, and version.
- Do not instruct untrained users to perform advanced procedures.
- Dosage or treatment changes are prohibited in generative chat.
- If a user describes urgent symptoms, direct them to local emergency services.
- Medical sharing is voluntary, granular, revocable, and purpose-specific.
- Full medical data is visible only to the data owner except an explicitly designed emergency-access flow.
- "HIPAA compliant" must not be claimed without a formal applicability assessment, BAAs, policies, controls, and audit. Apply strong health-data protections regardless.

11. Encryption Target

The current CryptoJS passphrase encryption is not the AES-256-GCM scheme claimed in existing architecture text.

Target:

- AES-256-GCM or managed envelope encryption.
- Random 96-bit nonce per encrypted value.

- Authentication tag stored with ciphertext.
- Additional authenticated data includes user ID, field name, and schema version.
- Key version stored per record.
- Master key in KMS/secret manager, never source or database.
- Rotation supports decrypt-old/encrypt-new.
- Decryption failures are audited and fail closed.

12. Threat Model And Controls

Threat	Required controls
Account takeover	rate limits, secure sessions, OAuth state, password policy, optional MFA later
IDOR/resource leakage	membership/ownership checks in every route and event
Socket impersonation	authenticated handshake; ignore client user IDs
Group invite abuse	high-entropy codes, expiry, rotation, join throttling
Medical disclosure	authenticated encryption, consent gates, minimal response, audit
Location stalking	opt-in sessions, expiry, revocation, no default history
Chat abuse	block/report, rate limits, moderation workflow, attachment scanning
Payment fraud	verified signatures, idempotent event table, entitlement from webhook state
LLM hallucination	schema validation, provider verification, source labels, fallback
SSRF/open redirect	allowlisted provider URLs, server-side URL construction
Upload abuse	MIME sniffing, size limits, image re-encode, malware scan, object storage
Log leakage	redaction, structured allowlist logging, restricted retention
Dependency compromise	lockfiles, automated scanning, reviewed updates

13. Retention

Define and implement configurable retention:

- Security/audit events: at least the operational/legal period.
- Chat: user-visible retention policy and deletion behavior.
- Exact location: ephemeral; SOS snapshot retained only as incident evidence.

- Medical profile: until user deletion or field removal, subject to legal hold.
- Provider cache: TTL based on source.
- Failed uploads and generation payloads: short-lived cleanup.
- Deleted account: anonymize shared trip/expense records where deletion would break other users' records.

06. Implementation Plan And Acceptance

1. Execution Principle

Work in vertical slices. A slice is complete only when UI, server validation, authorization, persistence, loading/error/empty states, tests, and documentation all pass.

2. Phase Plan

Phase 0: Baseline And Safety

- Preserve current user modifications.
- Add `.env.example`.
- Add health endpoints, standard error envelope, request IDs, and log redaction.
- Remove production client-side premium grant.
- Fix medical encryption claim and design migration.
- Add database constraints and migrations.
- Add demo seed and test accounts.

Exit: build/lint/tests pass; no known unauthenticated sensitive endpoint; no production privilege-escalation endpoint.

Phase 1: Account, Onboarding, And Shell

- App route group, loading/error states, settings placeholders.
- Resumable wizard and optional medical step.
- Object-storage avatar upload.
- Privacy/terms/safety pages.
- English message catalog and Hindi architecture.

Exit: new user can register through dashboard on mobile and desktop without medical disclosure.

Phase 2: Matching, Chat, And Groups

- Canonical compatibility formula and explanations.
- Pagination, filters, blocks/reports.

- Authenticated Socket.IO handshake.
- Pre-match chat and group chat hardening.
- Atomic group capacity/invites and group detail route.

Exit: two users can discover, chat, form/join a group, and cannot access another group's resources.

Phase 3: Trips And Discovery

- Versioned generation jobs and progress.
- Place verification/source status.
- Detail route, day timeline, map/list, votes and collaboration.
- Hidden-gem evidence and culinary filters.
- Dynamic replanning proposals.

Exit: group generates and collaboratively edits a feasible trip; prior versions remain recoverable.

Phase 4: Expenses, Safety, And Offline

- Integer-money expense model and settlements.
- Location-sharing sessions.
- Incident-based SOS and notification statuses.
- Reviewed first-aid content registry.
- Medication schedules and push notifications.
- Offline active-trip read cache.

Exit: expense and SOS drill tests pass; no unconsented medical or location access.

Phase 5: Billing, Operations, And Release

- Verified checkout and webhook entitlement lifecycle.
- Observability, retention jobs, export/deletion.
- E2E suite, load smoke tests, accessibility audit, incident runbook.
- Production deployment and rollback verification.

3. Definition Of Done

For every requirement:

- Code follows existing patterns and local Next.js 16 docs.
- Request and response are validated and typed.
- Authentication and resource authorization are tested.
- Database migration is reversible or has documented recovery.
- Loading, empty, error, offline, and success states exist.
- Keyboard and screen-reader behavior is checked.

- Analytics/logging excludes sensitive content.
- Unit and integration tests cover normal and failure behavior.
- User-visible data indicates live/cached/demo/unverified source.
- Documentation and API contract are updated.

4. Critical Acceptance Scenarios

AC-01 Registration And Optional Medical Data

Given a new adult user, when they register and complete required travel steps while skipping medical details, then they become onboarded and reach the dashboard. A minor or invalid input is rejected. No medical record is required.

AC-02 Resource Authorization

Given users A and B in different groups, when A requests B's group, messages, trip, expenses, medical overview, or socket room, then the server returns 403/neutral error and no resource data.

AC-03 Matching Explainability

Given two completed profiles, then the match response contains algorithm version, overall estimate, component scores, and plain-language reasons. Medical condition data does not affect ranking or appear in output.

AC-04 Atomic Group Join

Given a group with one remaining seat and two simultaneous valid join requests, then exactly one succeeds and member count never exceeds maximum.

AC-05 Chat Identity

Given a connected socket, sending a different client `userId` cannot impersonate that user. The server identity comes from the authenticated socket.

AC-06 Itinerary Verification

Given a generated itinerary with a place the provider cannot verify, then the item is saved/displayed as unverified and is never represented as a live confirmed place.

AC-07 Version Conflict

Given two members editing version 4, after one creates version 5, the other's stale update receives 409/conflict with current version and does not overwrite version 5.

AC-08 Regeneration Preservation

Regenerating a day or trip creates a new version. The previous version remains viewable and restorable.

AC-09 Hidden Gem Rule

A 3-day trip does not force hidden gems. A 5-day trip targets 30-40% verified gems but may return fewer with a visible explanation when evidence is insufficient.

AC-10 Money Integrity

For any accepted expense, split minor-unit amounts equal the total exactly. Non-members are rejected. Recalculated balances sum to zero.

AC-11 Medical Consent

Group member B cannot see A's emergency categories unless A's profile sharing and group-specific consent are both active. Revocation takes effect immediately and creates a consent event.

AC-12 Location Expiry

An expired/revoked location session rejects updates and stops broadcasts. The UI removes the sharing indicator only after server acknowledgment.

AC-13 SOS

Holding for less than three seconds does nothing. A completed hold creates one idempotent incident, immediately shows the emergency number, broadcasts to authorized members, and displays each notification attempt as sent, queued, or failed.

AC-14 Payment Entitlement

Calling an ordinary authenticated API cannot upgrade a user. A valid unique webhook updates entitlement once; replay does not duplicate state.

AC-15 Provider Failure

When maps, weather, OpenAI, or ML is unavailable, the page remains usable and clearly labels fallback/stale/unavailable data. It does not invent live information.

AC-16 Accessibility

All core journeys complete using keyboard only at 320 px and desktop widths. Focus is visible, dialogs trap/restore focus, map results have list equivalents, and reduced motion removes parallax.

5. Test Matrix

Unit

- Compatibility components and invariants.
- Embedding dimensions and value ranges.
- Group capacity and state transitions.
- Inclusive trip duration and timezone.
- Hidden-gem allocation.
- Money splits, remainders, balances, settlements.
- Consent predicate.
- Encryption round trip, tamper failure, and key version.
- First-aid content lookup.

Integration

- Registration/profile/medical routes.
- Every resource authorization boundary.
- Simultaneous group joins.
- Trip generation transaction and failure rollback.
- Itinerary optimistic concurrency.
- Expense transaction.
- Webhook verification/idempotency.
- Emergency incident and notification attempts.

Realtime

- Authenticated handshake.
- Unauthorized room join.
- Persist-before-broadcast.
- Duplicate client message ID.
- Stale itinerary update.
- Expired location session.

E2E

- Register -> onboard -> discover -> chat -> group -> trip.
- Existing group -> itinerary -> expense.
- Consent -> group overview -> revoke.
- SOS drill with denied and permitted location.
- Premium checkout in provider test mode.

- Offline active trip.

Non-Functional

- Axe or equivalent accessibility checks plus manual keyboard review.
- Lighthouse/core-web-vitals baseline.
- Dependency and secret scan.
- Authorization fuzz tests.
- Load smoke test for API and sockets; report measured hardware and configuration.
- Backup restore and migration rehearsal.

6. Current Baseline Verification

As of 2026-06-09:

- `npm run build`: passes on Next.js 16.2.6.
- `npm run lint`: passes.
- Jest: 1 suite, 2 tests pass.
- Python tests: not executed in the bundled runtime because `pytest` is not installed there. CI installs it separately; the implementation agent must run the service tests in a configured environment.

This is a weak coverage baseline, not release readiness.

7. Final Handoff Checklist

- No unresolved P0/P1 gap.
- All migrations applied to a clean database and an upgrade copy.
- Build, lint, unit, integration, E2E, Python, and accessibility checks pass.
- Demo mode is visibly labeled.
- Production secrets and provider restrictions reviewed.
- Medical content has documented professional review or is disabled.
- Backup restore and rollback tested.
- Final report includes metrics and known limitations without unsupported claims.

07. Current-State Gap Analysis

Snapshot: 2026-06-09

1. Executive Assessment

The repository is a working, buildable prototype with broad feature coverage. It is not yet the production-ready system described by the proposal. Many integrations and "models" are simulations or

heuristics, important screens are compressed into a few large client pages, security controls need hardening, and automated coverage is very small.

Antigravity should evolve this codebase. A rewrite would discard useful domain logic, migrations, routes, UI identity, and working fallbacks.

2. Implemented And Usable

- Next.js 16.2.6 App Router application with React 19 and Tailwind CSS 4.
- Warm editorial travel-journal design using EB Garamond, Manrope, rust/gold/cream palette.
- Credentials login, registration, optional Google provider configuration, session gating, and onboarding redirect.
- Travel/personality profile persistence and 18-dimensional embedding.
- Encrypted medical profile persistence and consent fields.
- Match discovery with component scores and conflict-risk call.
- Pre-match chat persistence and Socket.IO events.
- Travel groups, invite join, membership authorization helper, group details and messages.
- Trip generation, day/item persistence, Google Places or OSM enrichment, and deterministic fallback itineraries.
- Itinerary UI, hidden-gem presentation, map components, checklist, collaborative editor, and expense component.
- Safety dashboard, hold interaction, facility lookup, medical cards, first-aid protocols, medication checklist, and safety chat.
- PostgreSQL/Drizzle schema and migrations.
- FastAPI service with clustering, conflict, medical risk, gem, sentiment, and culinary endpoints.
- Dockerfiles, Docker Compose, GitHub Actions, Jest, Pytest test folder, Playwright configuration, and JMeter plan.
- Stripe/Razorpay webhook signature verification paths.
- Production build and lint pass.

3. Partial, Simulated, Or Misnamed

Area	Current behavior	Required correction
Conflict prediction	Mean feature difference	Label heuristic or implement evaluated Random Forest
Medical risk	Static rules	Label heuristic; clinical review; no ranking discrimination
Hidden gem	Weighted formula	Label heuristic or implement evaluated model and evidence

Area	Current behavior	Required correction
Sentiment	Keyword list	Moderation signal only; multilingual model only after evaluation
Weather	Simulated 24-hour pattern	Real provider/cache or unavailable label
Traffic	Coordinate-derived simulation	Real provider/cache or demo label
Events	Static festival/roadwork	Real source/cache or unavailable
Public transport	Static two-option response	Real directions provider or unavailable
Push reminders	Console logging	Persisted schedule, push subscription, worker, delivery status
Commission	Console logging	Modeled ledger only when booking integration exists
Location	Socket broadcast	Authenticated/time-limited sessions and expiry
Collaborative edit	Replaces first day's items	Versioned item-level operations across all days
Itinerary regeneration	Deletes existing group trips	Preserve versions and history
Expense math	Floating real; custom splits only persisted when supplied	Minor units, equal split generation, exact totals, audit
Avatar	Base64 stored in user field	Object storage, MIME/size validation, image processing
Offline	Service worker shell	Active-trip cache, stale indication, update strategy
Multilingual	Translation helper only	Message catalogs, locale routing, formatting
Premium	Direct <code>/users/upgrade</code> grant	Remove production path; webhook-derived entitlements

4. Critical Security And Safety Gaps

P0 Before Production

- Socket.IO trusts client-supplied user identity during room join before deriving a secure socket identity.
- `/api/users/upgrade` directly grants premium to any authenticated user.
- Current medical encryption uses CryptoJS passphrase AES and does not implement the claimed AES-256-GCM authenticated scheme.

- Group `update_location` and `emergency_sos` checks depend on prior socket state but require stronger group/session authorization.
- SOS facility lookup can run before confirming supplied `groupId` membership.
- Medical/safety chat can create liability if presented as expert medical advice.
- Simulated weather/traffic/facility information can be mistaken for live data.
- Exact authorization and rate-limit tests are missing across most endpoints.

P1

- Base64 avatar storage creates database and abuse risk.
- No block/report data model or moderation workflow.
- No consent audit trail, data export, deletion workflow, or retention jobs.
- Payment cancellation/downgrade and webhook idempotency are incomplete.
- No location sharing expiry/revoke model.
- No itinerary concurrency/versioning.
- Medical claims in README/architecture do not match implementation.
- External API timeouts, retries, circuit behavior, and source freshness are inconsistent.

5. Product And UX Gaps

- Onboarding is one large two-tab page rather than a resumable wizard.
- No dedicated group detail/chat routes in the page structure.
- Dashboard is a summary but lacks active-trip next-action and readiness depth.
- Discover lacks robust pagination/filtering, block/report, and careful score language.
- Some UI components use a generic gray/slate visual style instead of the journal design tokens.
- Marketing page uses several remote images and effects without a complete reduced-motion and resilient image strategy.
- Error/loading/empty/offline states are inconsistent.
- Accessibility has not been systematically verified.
- No privacy, terms, safety disclaimer, account settings, blocked users, export, or deletion screens.
- No evidence/source freshness pattern shared across itinerary, food, maps, weather, and facilities.

6. Data Gaps

- Monetary values use floating point.
- Trips have no itinerary versions.
- No audit events, consent events, reports, blocks, subscriptions, webhook events, location sessions, incidents, medication schedules/events, provider cache, or attachments.
- Medical encryption metadata/key version is absent.

- Invite expiry/rotation is absent.
- Compatibility cache does not include profile/algorithm version invalidation.
- Chat lacks client idempotency IDs and moderation state.
- Exact location retention is not modeled.

7. Architecture And Operations Gaps

- README says Next.js 15 while package is Next.js 16.2.6.
- Proposal describes Express API gateway, MongoDB, Redis, RabbitMQ, Kubernetes, and AWS components that do not exist and are not required for MVP.
- GitHub Actions uses Node 18 while the Docker build uses Node 20; align with supported runtime.
- No production readiness/health endpoints for the web service.
- No structured observability, audit logging, provider health, or error tracking.
- Cron scripts exist but production scheduling and distributed locking are unspecified.
- Socket.IO scaling strategy is absent.
- Docker build installs with `npm install` instead of deterministic `npm ci`.
- The standalone Next output is configured but the production image copies broad source/node_modules and launches through `tsx`; simplify and verify deployment strategy.

8. Test Gaps

Current automated evidence:

- Jest: one `MatchCard` suite with two tests.
- Python: one clustering test module exists.
- No committed Playwright E2E test files were found.
- JMeter plan exists but no recorded baseline result.

Highest-priority tests:

1. Authorization for every group/trip/chat/medical/expense route and socket event.
2. Group capacity race.
3. Medical consent grant/revoke.
4. Encryption tamper failure and rotation.
5. Itinerary transaction/version conflicts.
6. Expense exact-sum invariants.
7. Webhook signature and replay.
8. SOS idempotency, permission denial, and notification failure.

9. Existing User Changes

At snapshot time the worktree already contains modifications to:

- `docker-compose.yml`
- `src/app/onboarding/page.tsx`
- `src/lib/auth.ts`

The implementation agent must inspect and preserve these changes. Do not revert them as cleanup.

10. Recommended First Backlog

1. Remove direct premium upgrade and add webhook event/idempotency tables.
2. Authenticate Socket.IO handshake and remove trusted payload user IDs.
3. Add standard API errors, request IDs, authorization tests, and rate limits.
4. Implement medical encryption migration and consent audit.
5. Convert onboarding to resumable steps while preserving existing form data.
6. Add group detail route and block/report foundations.
7. Add itinerary versions/jobs instead of deleting previous trips.
8. Migrate expenses to minor units.
9. Label all simulations and provider freshness.
10. Add core E2E journey and safety drill tests.

08. Source Traceability

Source: Sahyatri 3. (1).pdf, 26 pages, supplied by the project owner.

1. Proposal-To-Spec Mapping

Proposal section/pages	Source intent	Authoritative destination
Abstract, pp. 2	Matchmaker-first integrated travel platform	PRD sections 1-3
Introduction, pp. 4	Matching, planning, medical safety, food, hidden gems, translation	PRD functional requirements
Problem, pp. 5	Fragmentation, conflict, health unpreparedness, overspend, authenticity	PRD goals and success metrics
Objectives, pp. 6-7	Matching, medical suite, planning, culinary, gems, navigation, budget, accessibility	PRD FR-MATCH through FR-I18N
Literature review, pp. 8-9	Claimed research gaps	Product rationale; claims require source validation before publication

Proposal section/pages	Source intent	Authoritative destination
Hypotheses, pp. 10-11	Improvement targets	PRD success metrics; treated as targets, not facts
Design/architecture, pp. 12	Microservices and cloud stack	Architecture; simplified to modular monolith + ML service for MVP
Matching method, pp. 13	Big Five, weighted compatibility, group constraints	Workflow canonical formula and ML rules
Planning/gems, pp. 14	Dijkstra, LLM, duration rules, gem scoring	Trip workflow, architecture, acceptance AC-06 to AC-09
Culinary/realtime/medical, pp. 15	Food matching, live context, emergency suite	PRD FR-DISC/FR-SAFE and workflow
UI/APIs, pp. 16	Key interfaces and provider list	UX screen specs and integration strategy
Data/testing, pp. 17	Synthetic data and testing targets	Implementation test matrix
Evaluation/ethics, pp. 18-19	Metrics, privacy, bias, medical disclaimer	AI lifecycle, safety, privacy, success metrics
MVP output, pp. 20-21	Feature and technical deliverables	MVP scope and final acceptance
Documentation/research, pp. 22-23	Documentation, paper, datasets, open source	Separate academic/research workstream; not a prerequisite for core app
UX/business/impact, pp. 23-25	Desired improvements, monetization, social impact	Metrics and future scope; no unsupported claims

2. Deliberate Corrections

Medical Profile Is Optional

The proposal calls medical profile creation mandatory. The build specification makes disclosure optional because health information is sensitive and consent must be meaningful. Safety readiness can encourage completion without blocking general travel use.

Medical Data Does Not Rank People

The proposal includes "medical compatibility" in matching. The target does not penalize or exclude people based on health conditions. It uses a separate emergency-sharing readiness signal and destination-specific warnings.

Architecture Is Incremental

The proposal requests PostgreSQL, MongoDB, Redis, RabbitMQ, Kubernetes, and many microservices. The current repository successfully uses Next.js, FastAPI, and PostgreSQL. Additional infrastructure is deferred until measured needs justify it.

Models Are Named Honestly

Current conflict, medical risk, gem, and sentiment code is heuristic. It must not be described as Random Forest, XGBoost, or BERT until trained artifacts and evaluations exist.

Claims Require Evidence

Targets such as 85% accuracy, 99.9% uptime, 10,000 concurrent users, 35% faster emergency response, and HIPAA compliance are not accepted as current facts. They require a documented test or compliance program.

Hidden Gem Allocation

The proposal alternates between 40% gems and a 40/30/30 split with different meanings. The authoritative requirement is a 30-40% target for verified lesser-known experiences on trips of 5 or more days, with no fabrication to meet a quota.

Safety Content Is Not Generative Authority

The proposal suggests tailored first aid and AI chat. The build specification requires static, versioned, reviewed emergency cards and restricts generative chat from diagnosis or treatment changes.

3. Repository-To-Spec Mapping

Existing area	Main target document
src/app, src/components, src/app/globals.css	UX/UI
src/db/schema.ts, migrations	Data/API
src/app/api	Data/API and architecture
server.ts, src/hooks/useSocket.ts	Realtime contract and security
src/lib/matching, FastAPI models	Workflows/AI
src/lib/itinerary, map/weather/traffic services	Workflows/architecture
src/lib/medical, safety page	Safety/security
Expense modules/routes/components	PRD and acceptance
Docker/Compose/GitHub Actions	Architecture and implementation

4. Open Product Decisions

The following may be decided during implementation but must be recorded in an ADR and must not weaken acceptance criteria:

- Exact identity verification provider.
- Primary maps/weather/translation vendors by deployment region.
- Free and premium quotas/prices.
- Group voting threshold.
- Exact medical content reviewer and review cycle.
- Data retention periods by jurisdiction.
- Whether exact SOS location is retained and for how long.
- Production hosting vendor.

5. Final Instruction To The Build Agent

Do not interpret the proposal as permission to fabricate integrations, model performance, live data, medical authority, or production scale. Build the smallest honest system that satisfies the authoritative requirements, preserve the existing product identity, and make every claim testable.